

CS3002E Compiler Design

Monsoon 2025

Lecture #10

Top-Down Parsing-3

Department of Computer Science & Engineering
NIT Calicut

Computing FIRST¹

1. For a terminal a , $FIRST(a) = \{a\}$.
2. If $X \rightarrow \epsilon$, add ϵ to $FIRST(X)$.
3. For a string α ,
 - 3.1 If α is of the form $a\alpha'$, then $FIRST(\alpha) = \{a\}$.
 - 3.2 If α is of the form $X_1X_2..X_k$, $k \geq 1$,
 - 3.2.1 add everything in $FIRST(X_1)$, except ϵ to $FIRST(\alpha)$.
 - 3.2.2 if for some i , $FIRST(X_1X_2..X_i)$ contains ϵ , add everything in $FIRST(X_{i+1})$, except ϵ to $FIRST(\alpha)$.
 - 3.2.3 if for all $i = 1 \dots k$, $FIRST(X_i)$ contains ϵ , then add ϵ to $FIRST(\alpha)$.
4. If $A \rightarrow \alpha$, add everything in $FIRST(\alpha)$ to $FIRST(A)$.

¹Refer ALSU 4.4.2: Algorithm to compute FIRST(X) for all grammar symbols X

Computing FOLLOW: Example

$$S \rightarrow (A)$$

$$A \rightarrow id \mid num \mid \epsilon$$

$) \in FOLLOW(A)$ because of $S \rightarrow (A)$

$$S\$ \Rightarrow (A)\$ \Rightarrow ()\$$$

For node A, parser should choose $A \rightarrow \epsilon$, when the lookahead is $)$.

If $A \rightarrow \alpha B \beta$, add everything in $FIRST(\beta)$ except ϵ to $FOLLOW(B)$.

Computing FOLLOW: Example

$$S \rightarrow idA$$

$$A \rightarrow +id \mid \epsilon$$

$$S\$ \Rightarrow idA\$ \Rightarrow id\$$$

$\$ \in FOLLOW(S)$, and hence $\$ \in FOLLOW(A)$

For node A, Parser should choose $A \rightarrow \epsilon$ when the lookahead is \$.

Computing FOLLOW: Example

$$S \rightarrow (A)$$
$$A \rightarrow idB$$
$$B \rightarrow +id \mid \epsilon$$
$$S\$ \Rightarrow (A)\$ \Rightarrow (idB)\$ \Rightarrow (id)\$$$

) $\in FOLLOW(B)$, because of the sentential form (idB).

Since $A \rightarrow idB$, everything in $FOLLOW(A)$ is in $FOLLOW(B)$.

For node B, parser should choose $B \rightarrow \epsilon$ when lookahead is).

If $A \rightarrow \alpha B$, add everything in $FOLLOW(A)$ to $FOLLOW(B)$.

Computing FOLLOW

To compute $FOLLOW(A)$ for all nonterminals A , apply the following rules until nothing more can be added to any of the $FOLLOW$ sets:

1. Place \$ in $FOLLOW(S)$ where S is the start symbol.
2. If there is a production $A \rightarrow \alpha B \beta$, add everything in $FIRST(\beta)$, except ϵ to $FOLLOW(B)$.
3. If there is a production $A \rightarrow \alpha B$, add everything in $FOLLOW(A)$ to $FOLLOW(B)$
4. If there is a production $A \rightarrow \alpha B \beta$, such that $FIRST(\beta)$ contains ϵ , add everything in $FOLLOW(A)$ to $FOLLOW(B)$.

Predictive Parsing Table Construction

For each production $A \rightarrow \alpha$

1. for each terminal $a \in FIRST(\alpha)$, add $A \rightarrow \alpha$ to $T[A, a]$.
2. if ϵ is in $FIRST(\alpha)$, for each terminal b in $FOLLOW(A)$, add $A \rightarrow \alpha$ to $T[A, b]$.
3. if ϵ is in $FIRST(\alpha)$, and $\$$ is in $FOLLOW(A)$, add $A \rightarrow \alpha$ to $T[A, \$]$.

Build Predictive Parsing Table

$$S \rightarrow AB$$

$$A \rightarrow aA \mid \epsilon$$

$$B \rightarrow bB \mid \epsilon$$

Build Predictive Parsing Table

$S \rightarrow (A)$

$A \rightarrow BC$

$B \rightarrow b \mid \epsilon$

$C \rightarrow c \mid \epsilon$

Build Predictive Parsing Table

$$A \rightarrow BC$$

$$B \rightarrow b \mid \epsilon$$

$$C \rightarrow c \mid \epsilon$$

Predictive Parsing Table

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow id$$

	id	+	\$
E	$E \rightarrow TE'$		
T	$T \rightarrow id$		
E'		$E' \rightarrow +TE'$	$E' \rightarrow \epsilon$

Non recursive predictive parsing

Input: string w and Parsing Table T for Grammar G

Output: if $w \in L(G)$, productions in a leftmost derivation of w ;
otherwise error indication

- Stack of grammar symbols - initially S on top of $\$$
- Input Buffer - $w\$$

Parser Configurations

	id	+	\$
E	$E \rightarrow TE'$		
T	$T \rightarrow id$		
E'		$E' \rightarrow +TE'$	$E' \rightarrow \epsilon$

Stack	Input	Action
$\leftarrow$ top		
$E\$$	$id + id\$$	$E \rightarrow TE'$
$TE'\$$	$id + id\$$	$T \rightarrow id$
$idE'\$$	$id + id\$$	match id
$E'\$$	$+id\$$	$E' \rightarrow +TE'$
$+TE'\$$	$+id\$$	match $+$
$TE'\$$	$id\$$	$T \rightarrow id$
$idE'\$$	$id\$$	match id
$E'\$$	$\$$	$E' \rightarrow \epsilon$
$\$$	$\$$	accept

Error Detection

Show error detection with inputs:

$id++$

$id+id+$

Reference:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.

Further reading:

- ALSU chapter 4, sections 4.4.2, 4.4.4